



Progetto di Laboratorio Interdisciplinare B

Università degli Studi dell'Insubria - Dipartimento di Scienze Teoriche e Applicate

Autori

Gharsellaoui Omema - 761146 - VA

Kufura Razak - 760614 - VA

Anno Accademico 2024/2025

Data: Giugno 2026 - Versione 1.0

Indice

1. Introduzione	3
2. Architettura del Sistema	3
3. Progettazione del Database	4
3.0 Analisi e Ristrutturazione dei Requisiti	4
3.1 Diagramma ER non ristrutturato	5
3.1b Diagramma ER ristrutturato	6
3.2 Vincoli di Integrità	6
3.3 Schema Relazionale	7
3.4 Conversione del Prezzo	8
4. Progettazione del Software (UML)	8
4.1 Use Case Diagram	9
4.2 Class Diagram	10
4.3 Sequence Diagram	11
4.4 State Diagram	15
4.5 Communication Diagram	16
5. Pattern Utilizzati	17
6. Strutture Dati	19
7. Scelte Algoritmiche	20
8. Gestione della Concorrenza	21
9. Interfaccia Grafica.....	24
10. JavaDoc	26
11. Bibliografia e Sitografia	27

1. Introduzione:

TheKnife è una piattaforma client/server, sviluppata in linguaggio Java, che permette agli utenti di cercare ristoranti in tutto il mondo, gestire una lista personale di preferiti ed inserire, modificare o cancellare recensioni. L'applicazione è ispirata al servizio TheFork e si propone come laboratorio applicativo dei concetti appresi nei corsi di Progettazione del Software, Basi di Dati e Sistemi Concorrenti e Distribuiti.

Il sistema è composto da due moduli principali:

- ServerTK — modulo server che interagisce con un database PostgreSQL e fornisce i servizi di back-end.
- ClientTK — modulo client con interfaccia grafica Swing che mette a disposizione tutti i servizi e le funzionalità per gli utilizzatori.

L'applicazione consente a:

- Utenti non registrati (guest) di visualizzare ristoranti e recensioni in forma anonima.
- Clienti registrati di gestire preferiti e recensioni (inserimento, modifica, cancellazione).
- Ristoratori registrati di inserire ristoranti, visualizzare statistiche sulle recensioni e rispondere ad esse.

2. Architettura del Sistema

TheKnife adotta un'architettura distribuita di tipo client/server organizzata su tre livelli logici (Presentazione, Logica Applicativa, Persistenza Dati). La separazione dei livelli consente di isolare la GUI dalla logica di business e dal database, favorendo manutenibilità ed estendibilità.

Componenti Principali:

Componente	Descrizione
ClientTK	Interfaccia utente Swing. Si connette al server tramite socket sulla porta 12345.
ServerTK	Server centrale. Resta in attesa di connessioni dai client mediante ServerSocket.
SlaveThread	Thread dedicato per ogni client connesso. Gestisce la comunicazione e le richieste.
DataBaseManager	Gestisce la connessione al database PostgreSQL tramite JDBC.
PostgreSQL	Database relazionale che memorizza utenti, ristoranti, recensioni, preferiti e risposte.

Diagramma architetturale

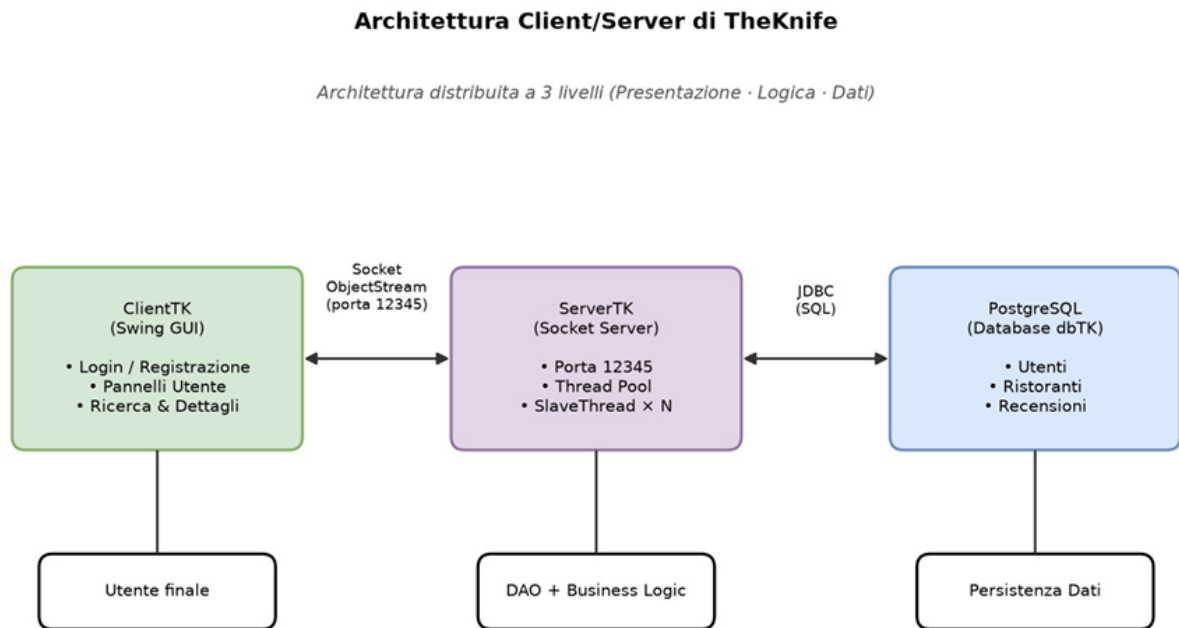


Figura 1 — Architettura a tre livelli di TheKnife.

Flusso di comunicazione

- Il client si connette al server tramite Socket su localhost:12345.
- Il server accetta la connessione e crea un SlaveThread dedicato (uno per ogni client).
- Client e server comunicano scambiandosi oggetti serializzati tramite ObjectOutputStream / ObjectInputStream.
- Il server interagisce con il database PostgreSQL tramite il livello DAO (Data Access Object).
- Le risposte vengono serializzate e re-inviolate al client, che le interpreta nella GUI Swing

3. Progettazione del Database:

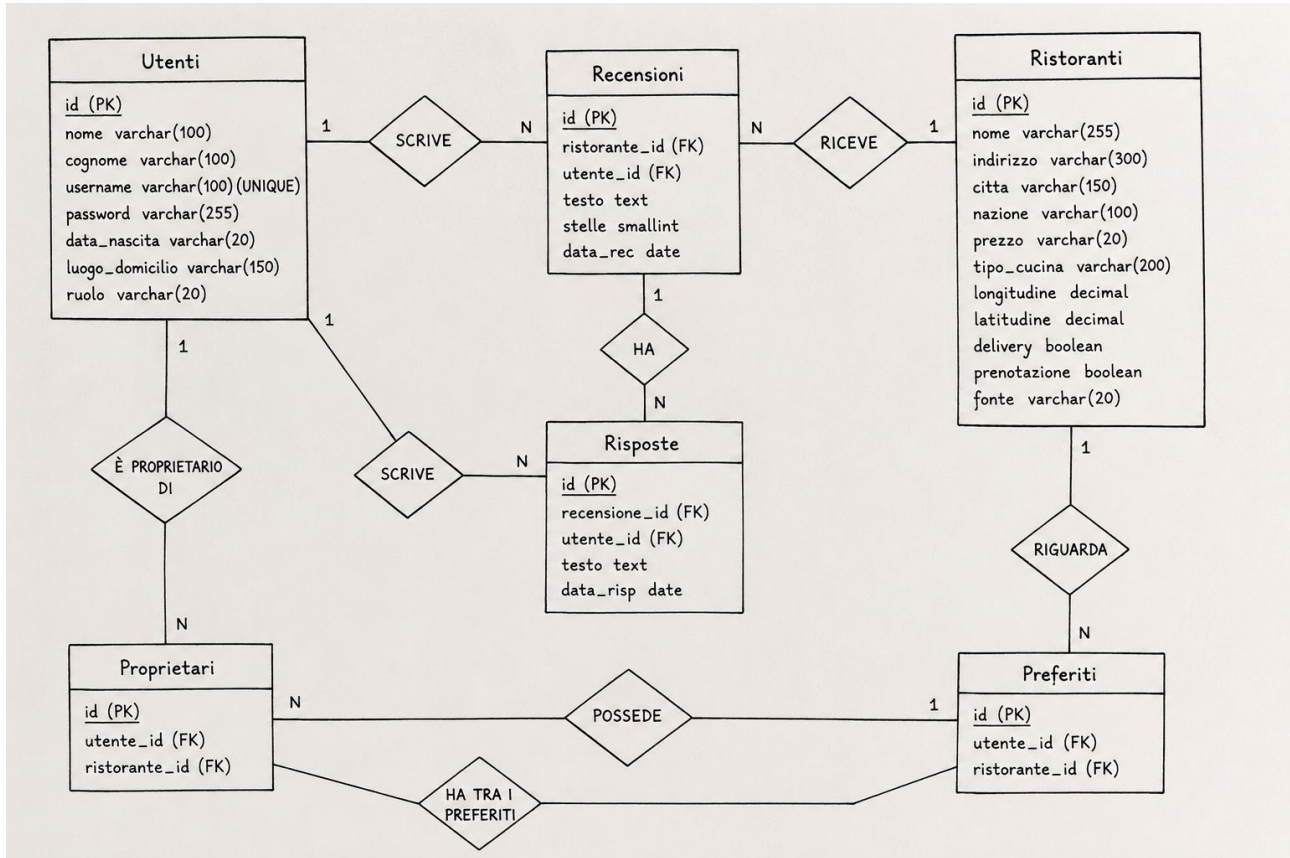
Il database, denominato dbTK, è realizzato in PostgreSQL ed è progettato per supportare in modo efficiente tutte le operazioni richieste dall'applicazione (ricerca multi-criterio, gestione recensioni, preferiti, ecc.). La progettazione è partita da un'analisi delle entità principali individuate nei requisiti, seguita da una ristrutturazione concettuale ed una traduzione nello schema relazionale.

3.0 Analisi e Ristrutturazione dei Requisiti

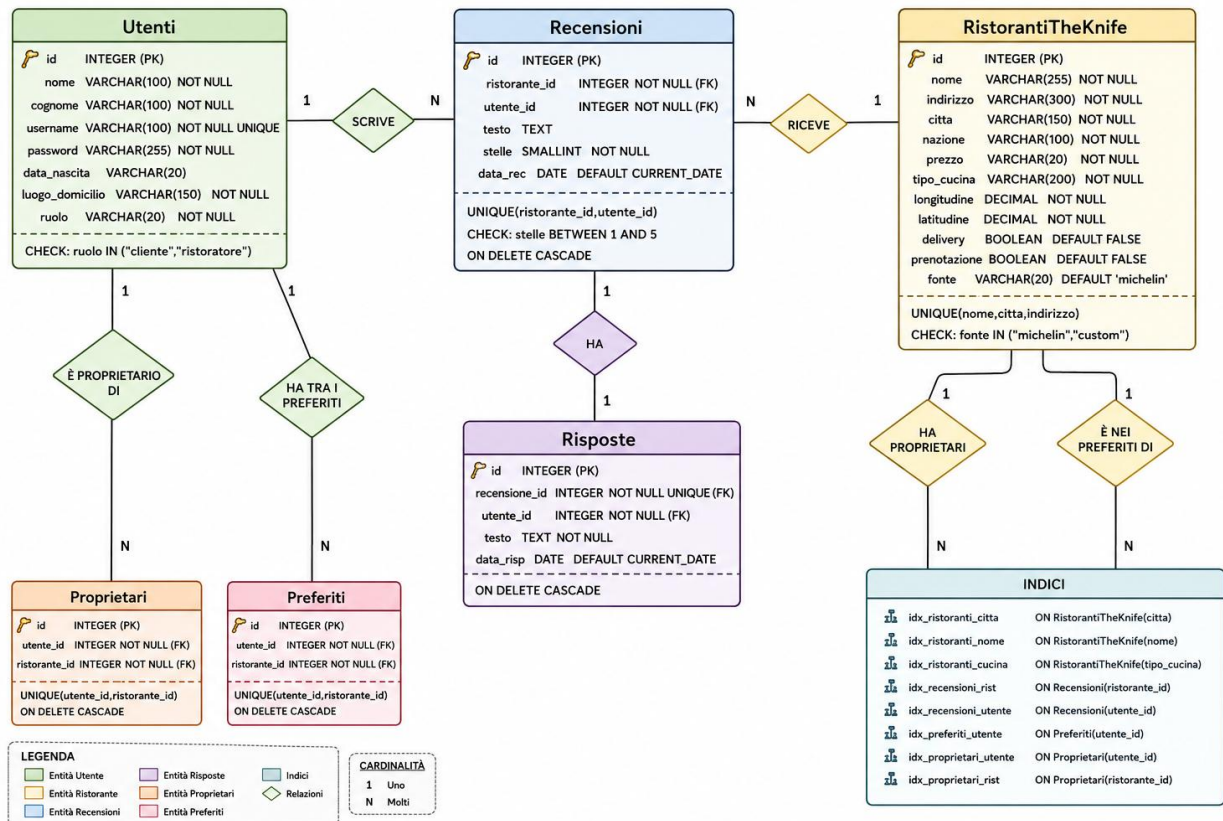
- Utenti raccoglie clienti e ristoratori distinguendoli tramite il campo ruolo.
- RistorantiTheKnife contiene dati anagrafici, geografici e servizi del ristorante.
- Recensioni registra stelle, testo e data della recensione.
- Preferiti e Proprietari materializzano relazioni molti-a-molti.

- Risposte è separata da Recensioni per rispettare il vincolo 0..1 risposta per recensione.

3.1 Diagramma ER non ristrutturato:



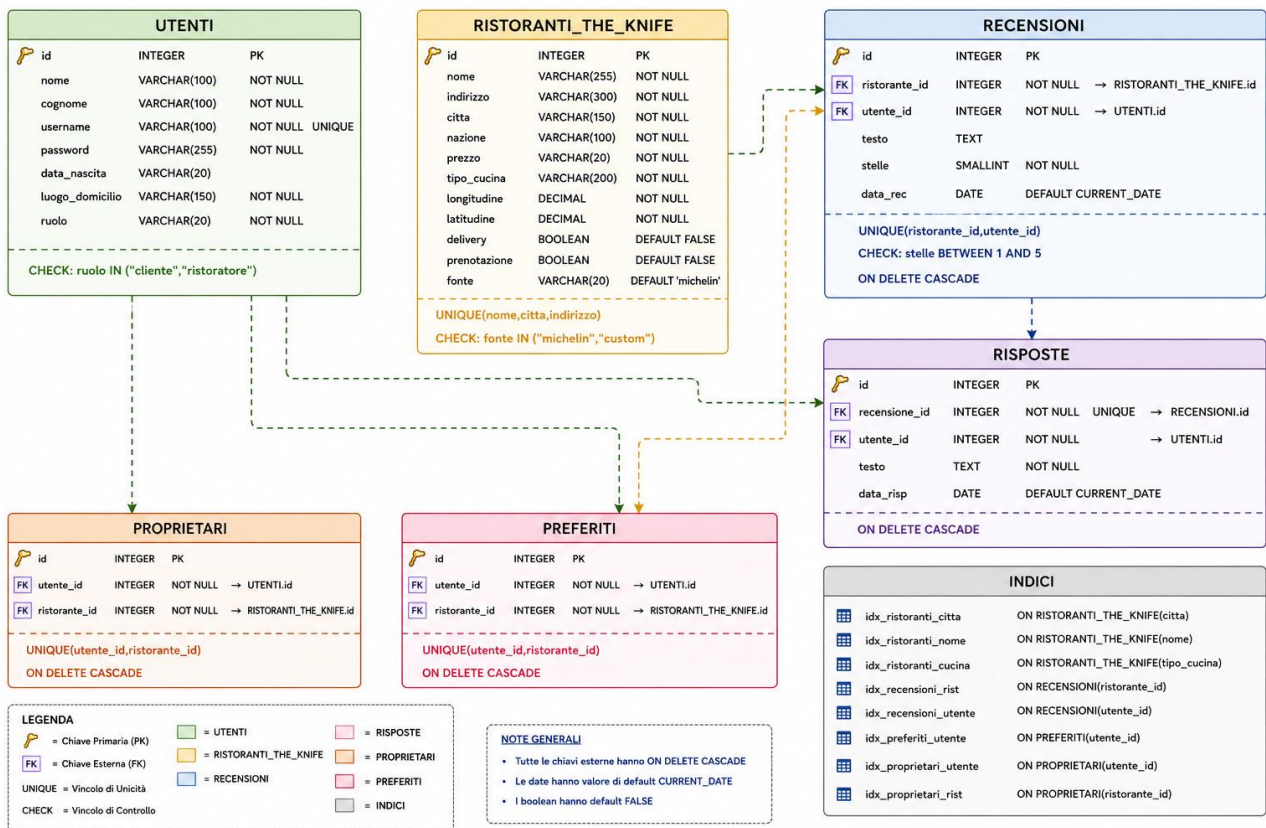
3.1 Diagramma ER ristrutturato:



3.2 Vincoli di Integrità:

Vincolo	Tabella	Descrizione
UNIQUE(username)	Utenti	Lo username deve essere univoco.
UNIQUE(ristorante_id, utente_id)	Recensioni	Un cliente può recensire un ristorante una sola volta.
UNIQUE(utente_id, ristorante_id)	Preferiti	Un ristorante può essere salvato una sola volta nei preferiti.
UNIQUE(recensione_id)	Risposte	Una recensione può avere al massimo una risposta.
CHECK(stelle BETWEEN 1 AND 5)	Recensioni	Il voto deve essere compreso tra 1 e 5.
CHECK(ruolo IN ...)	Utenti	Il ruolo deve essere 'cliente' o 'ristoratore'.
FK ... ON DELETE CASCADE	Tutte	Eliminazione a cascata per integrità referenziale.

3.3 Schema Relazionale:



Script SQL essenziale:

```
-- TABELLA: Utenti
CREATE TABLE Utenti (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    cognome VARCHAR(100) NOT NULL,
    username VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    data_nascita VARCHAR(20),
    luogo_domicilio VARCHAR(150) NOT NULL,
    ruolo VARCHAR(20) NOT NULL
    CHECK (ruolo IN ('cliente', 'ristoratore'))
);

-- TABELLA: RistorantiTheKnife
CREATE TABLE RistorantiTheKnife (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(255) NOT NULL,
    indirizzo VARCHAR(300) NOT NULL,
    citta VARCHAR(150) NOT NULL,
    nazione VARCHAR(100) NOT NULL,
    prezzo VARCHAR(20) NOT NULL,
    tipo_cucina VARCHAR(200) NOT NULL,
    longitudine DOUBLE PRECISION NOT NULL,
    latitudine DOUBLE PRECISION NOT NULL,
    delivery BOOLEAN NOT NULL DEFAULT FALSE,
    prenotazione BOOLEAN NOT NULL DEFAULT FALSE,
    fonte VARCHAR(20) NOT NULL DEFAULT 'michelin'
    CHECK (fonte IN ('michelin', 'custom')),
    UNIQUE (nome, citta, indirizzo)
);
```

3.4 Conversione del Prezzo:

Il campo prezzo nella tabella RistorantiTheKnife memorizza i prezzi sia in formato simbolico (es. €€€€ per i ristoranti Michelin) sia in formato numerico (per i ristoranti inseriti manualmente dai ristoratori). Per consentire ricerche per fascia di prezzo, il sistema converte i simboli in valori numerici mediante la seguente logica SQL:

```
CASE
  WHEN prezzo ~ '^[0-9]+(\.[0-9]+)?$' THEN prezzo::DOUBLE PRECISION
  ELSE LENGTH(TRIM(prezzo)) * 37.5
END
```

Mappatura dei simboli

Simbolo	Valore numerico (€)
€	37,5 €
€€	75 €
€€€	112,5 €
€€€€	150 €
€€€€€	187,5 €

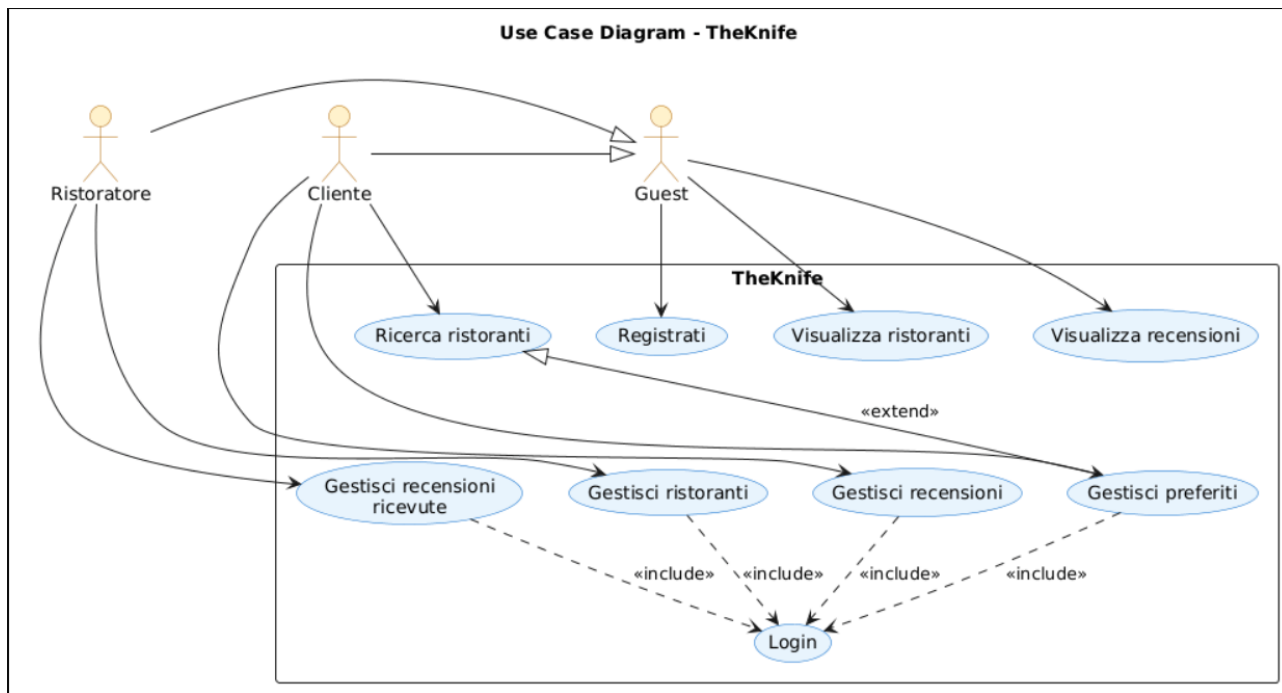
Esempio: se un ristorante ha prezzo €€€€, la conversione restituisce $4 \times 37,5 = 150$ €. Questo permette di filtrare i ristoranti anche quando l'utente inserisce un intervallo numerico (es. "da 80 € a 120 €").

4. Progettazione del Software (UML):

La progettazione del software è documentata mediante diagrammi UML che ne descrivono la struttura statica (Class Diagram, Use Case Diagram) e dinamica (Sequence Diagram, State Diagram, Communication Diagram).

4.1 Use Case Diagram:

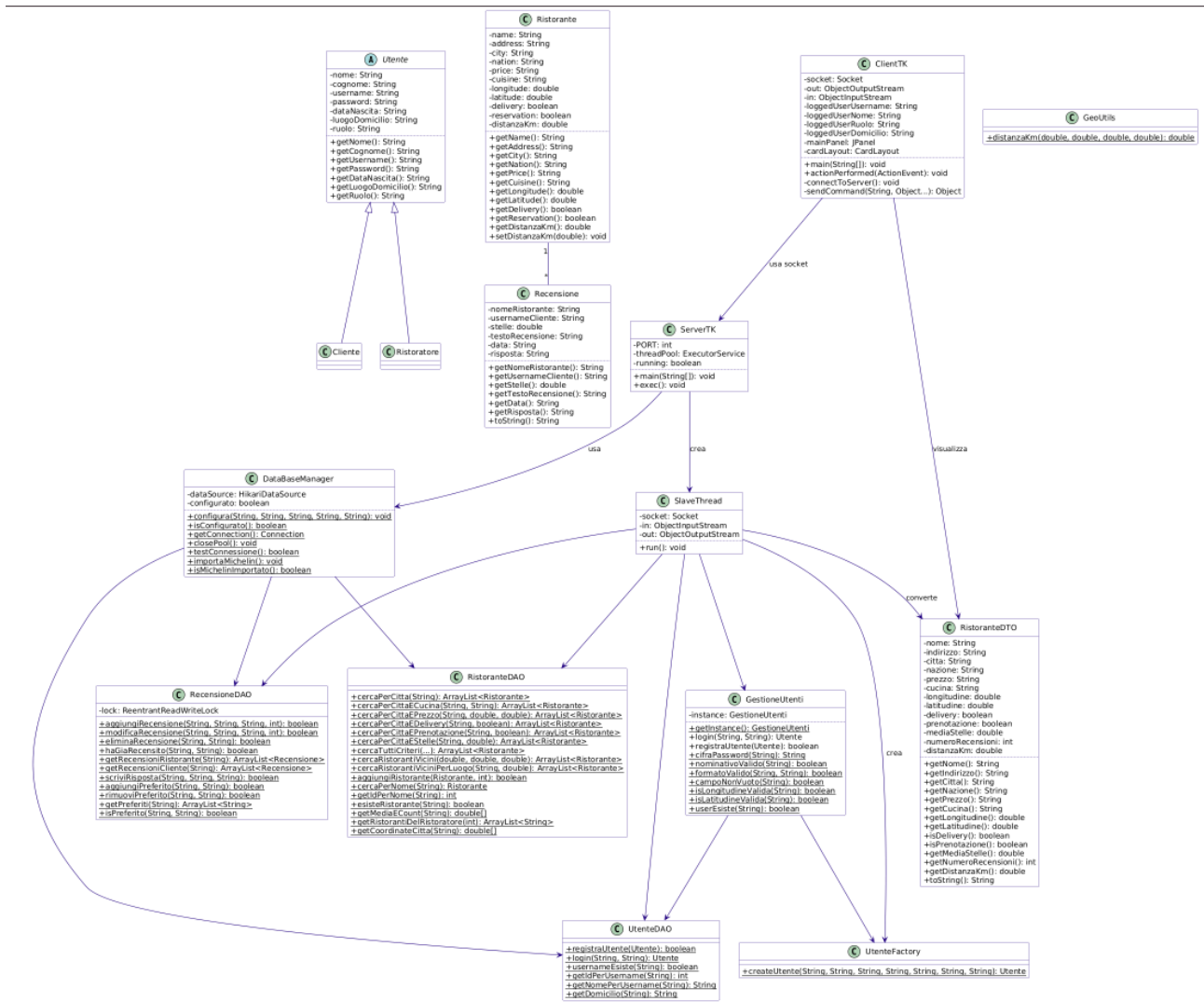
Il diagramma dei casi d'uso identifica i tre attori del sistema (Guest, Cliente, Ristoratore) e le funzionalità accessibili a ciascuno. Le funzionalità di sola lettura sono disponibili senza autenticazione; le altre richiedono il login.



Il Guest può visualizzare ristoranti, consultare recensioni e registrarsi. Il Cliente gestisce preferiti e recensioni. Il Ristoratore gestisce i propri ristoranti e le recensioni ricevute.

4.2 Class Diagram:

Il diagramma delle classi mostra le entità di dominio (Utente, Cliente, Ristoratore, Ristorante, Recensione), il livello DAO, le classi di gestione (manager) e le utility. La classe astratta Utente è specializzata in Cliente e Ristoratore tramite ereditarietà



Descrizione delle classi principali:

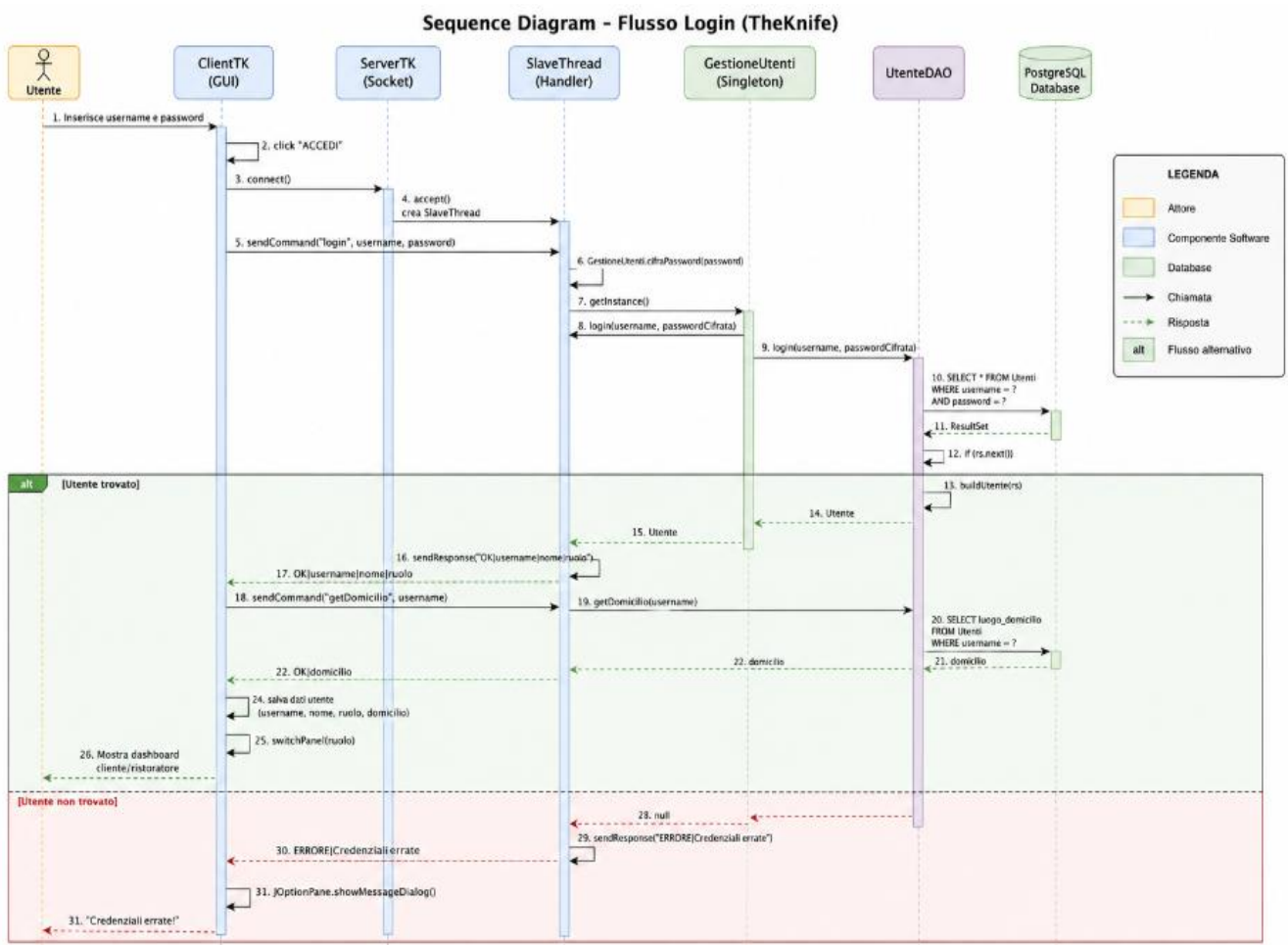
Classe	Tipo	Descrizione
ClientTK	JFrame	Interfaccia grafica Swing. Contiene i pannelli (login, registrazione, guest, cliente, ristorante, ricerca).
ServerTK	—	Avvia il server socket sulla porta 12345. Crea uno SlaveThread per ogni client.
SlaveThread	Thread	Gestisce la comunicazione con un singolo client. Riceve comandi, interagisce con i DAO, restituisce risposte.
DataBaseManager	—	Gestisce la connessione al database PostgreSQL. Importa il dataset Michelin.
Utente	abstract	Classe base per Cliente e Ristorante.
Cliente	extends Utente	Cliente registrato. Ruolo: 'cliente'.
Ristoratore	extends Utente	Ristoratore registrato. Ruolo: 'ristoratore'.
Ristorante	—	Entità di dominio del ristorante.

Recensione	—	Entità della recensione (con eventuale risposta).
RistoranteDTO	Serializable	Data Transfer Object per il trasferimento su rete.
UtenteDAO	DAO	Operazioni CRUD su Utenti.
RistoranteDAO	DAO	Operazioni CRUD su Ristoranti.
RecensioneDAO	DAO	Operazioni CRUD su Recensioni, Risposte, Preferiti.
GestioneUtenti	Singleton	Gestisce la registrazione e il login.
GestioneRistoranti	—	Gestisce le ricerche e la visualizzazione dei ristoranti.
GeoUtils	—	Utility per il calcolo delle distanze geografiche (Haversine).

4.3 Sequence Diagram:

I sequence diagram seguenti illustrano i tre scenari più rilevanti del sistema: l'autenticazione dell'utente, la ricerca multi-criterio dei ristoranti e l'inserimento di una nuova recensione. Le frecce continue indicano messaggi sincroni; le tratteggiate indicano i ritorni.

4.3.1 Flusso di Login:

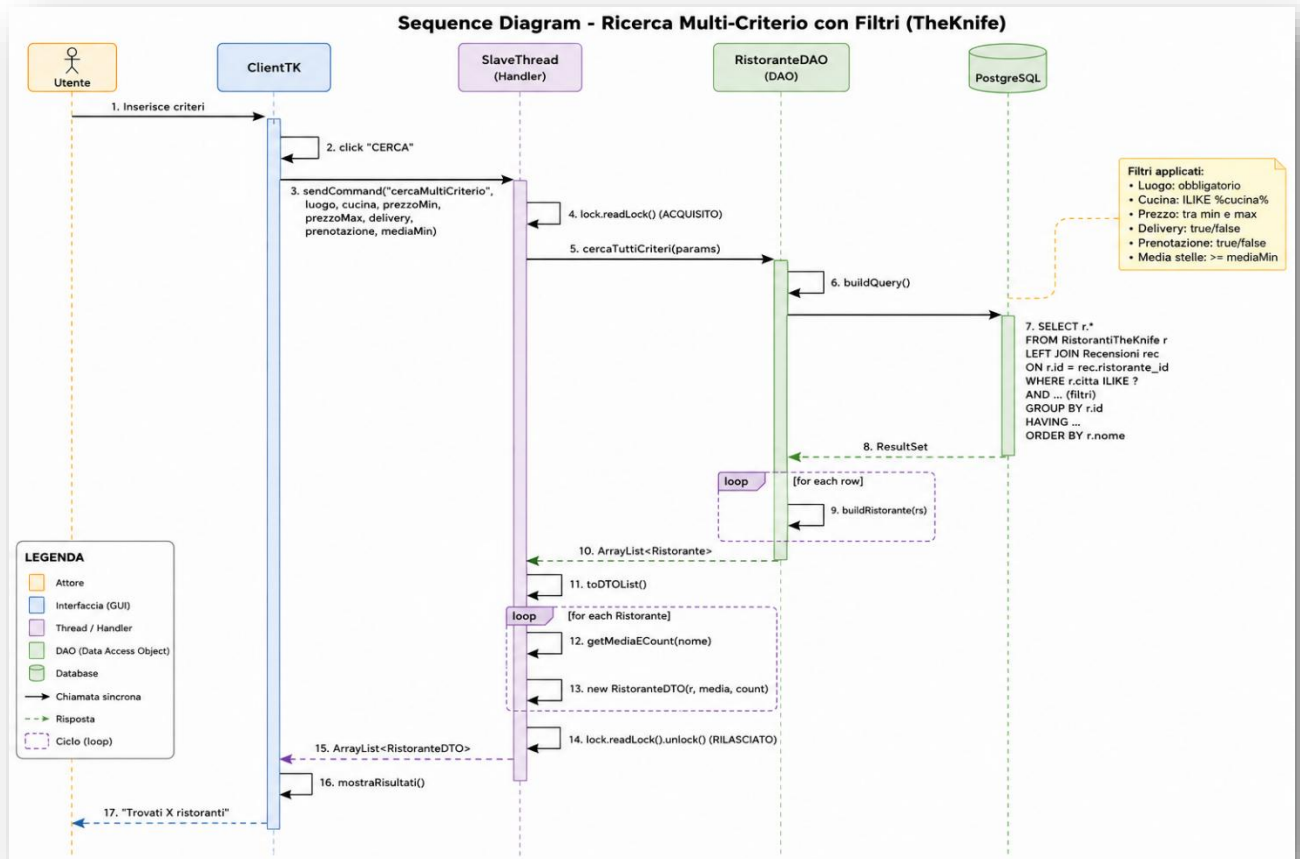


Il flusso si sviluppa nelle seguenti fasi:

- L'utente inserisce username e password e preme il pulsante "**ACCEDI**" sull'interfaccia ClientTK.
- Il client apre una connessione socket con il server e invia il comando di login contenente le credenziali.
- Il server accetta la connessione e crea uno **SlaveThread**, dedicato esclusivamente alla gestione di quel client.
- Lo SlaveThread ottiene l'istanza di **GestioneUtenti** (implementata mediante il pattern Singleton), cifra la password con l'algoritmo **SHA-256** e avvia la procedura di autenticazione.
- **GestioneUtenti** delega la verifica delle credenziali a **UtenteDAO**, che esegue una query SQL sul database PostgreSQL.
- Se il database trova una corrispondenza, viene costruito l'oggetto **Utente** (Cliente o Ristoratore) tramite il **Factory Method** buildUtente().
- Lo SlaveThread invia quindi al client una risposta contenente username, nome e ruolo dell'utente autenticato.
- Successivamente il client richiede anche il domicilio dell'utente, che viene recuperato dal database e restituito al client.
- Una volta ricevuti tutti i dati, il client memorizza le informazioni della sessione e visualizza automaticamente il pannello corrispondente al ruolo dell'utente (Cliente o Ristoratore).

Nel caso in cui le credenziali non siano corrette, il database non restituisce alcun risultato; il valore **null** viene propagato fino allo SlaveThread, che invia al client un messaggio di errore. L'interfaccia grafica mostra quindi una finestra di dialogo con l'avviso "**Credenziali errate!**" senza consentire l'accesso al sistema.

4.3.2 Ricerca Multi-Criterio:



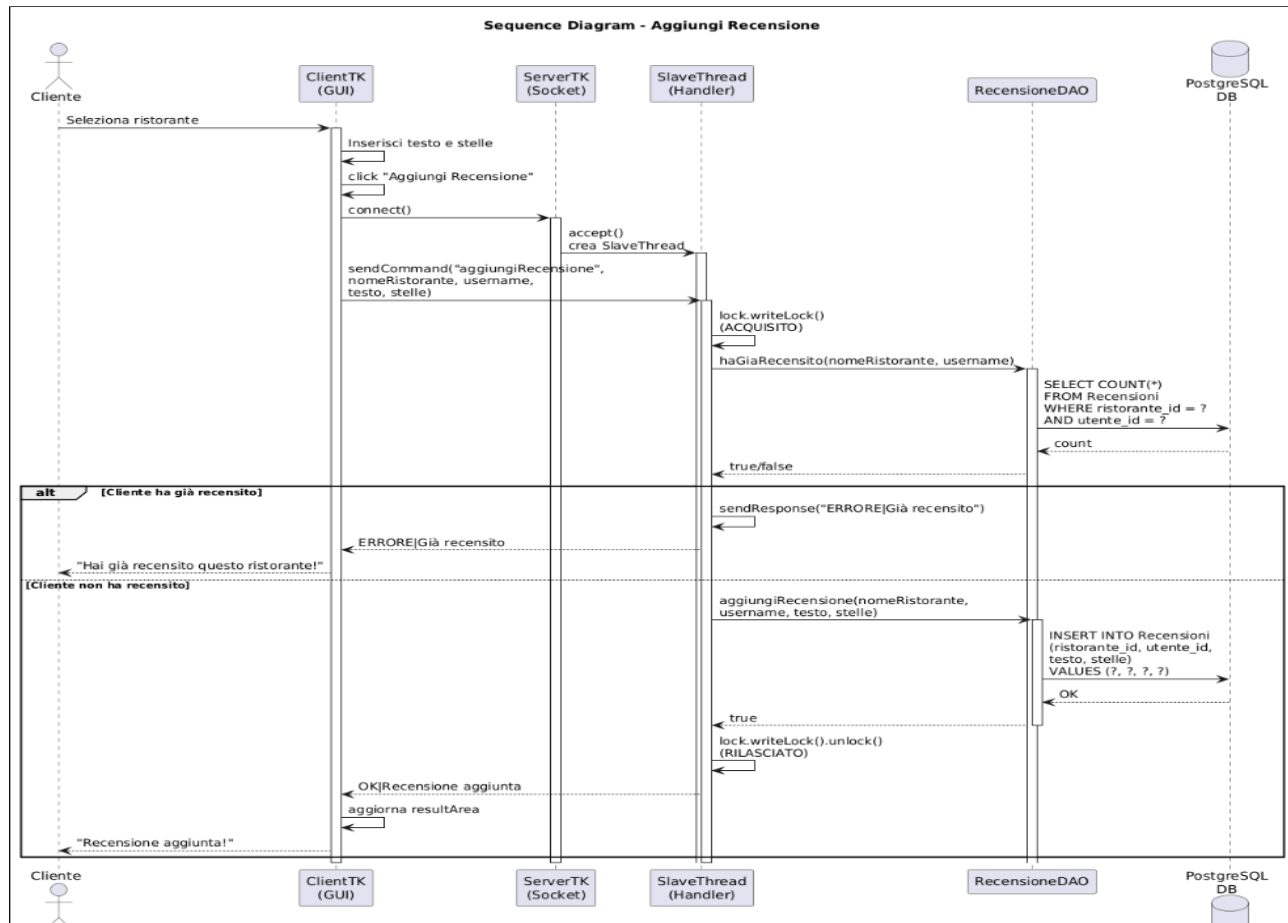
Descrizione:

Il Sequence Diagram rappresenta il flusso della ricerca multi-criterio dei ristoranti. L'utente inserisce i filtri desiderati (luogo, cucina, fascia di prezzo, disponibilità del delivery, prenotazione e valutazione minima) e il client invia la richiesta allo **SlaveThread**.

Lo **SlaveThread** acquisisce un lock in lettura e delega l'operazione al **RistoranteDAO**, che costruisce dinamicamente la query SQL applicando solo i filtri selezionati dall'utente. La query viene eseguita sul database PostgreSQL e, per ogni risultato ottenuto, viene creato un oggetto **Ristorante**.

Successivamente lo **SlaveThread** converte gli oggetti in **RistoranteDTO**, calcola le informazioni aggiuntive (media delle recensioni e numero di recensioni), rilascia il lock e restituisce la lista dei risultati al client. Infine il client visualizza l'elenco dei ristoranti trovati all'utente

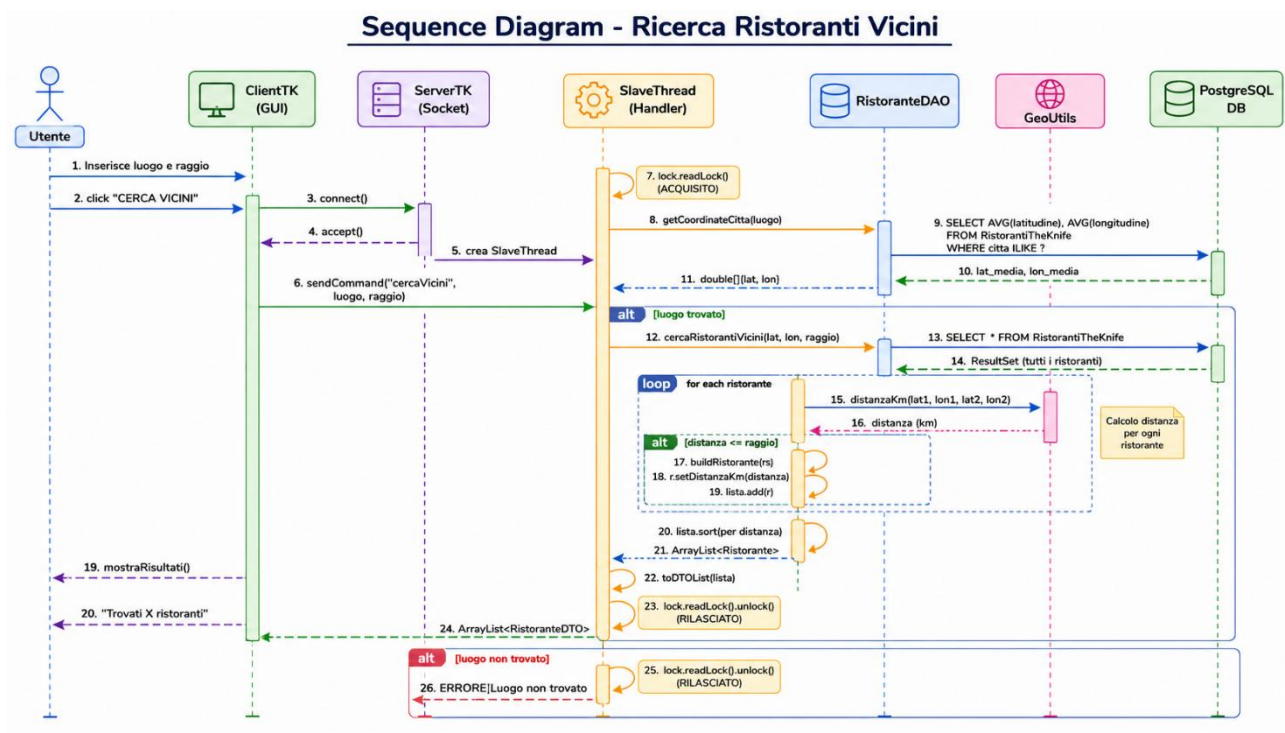
4.3.3 Aggiunta Recensione:



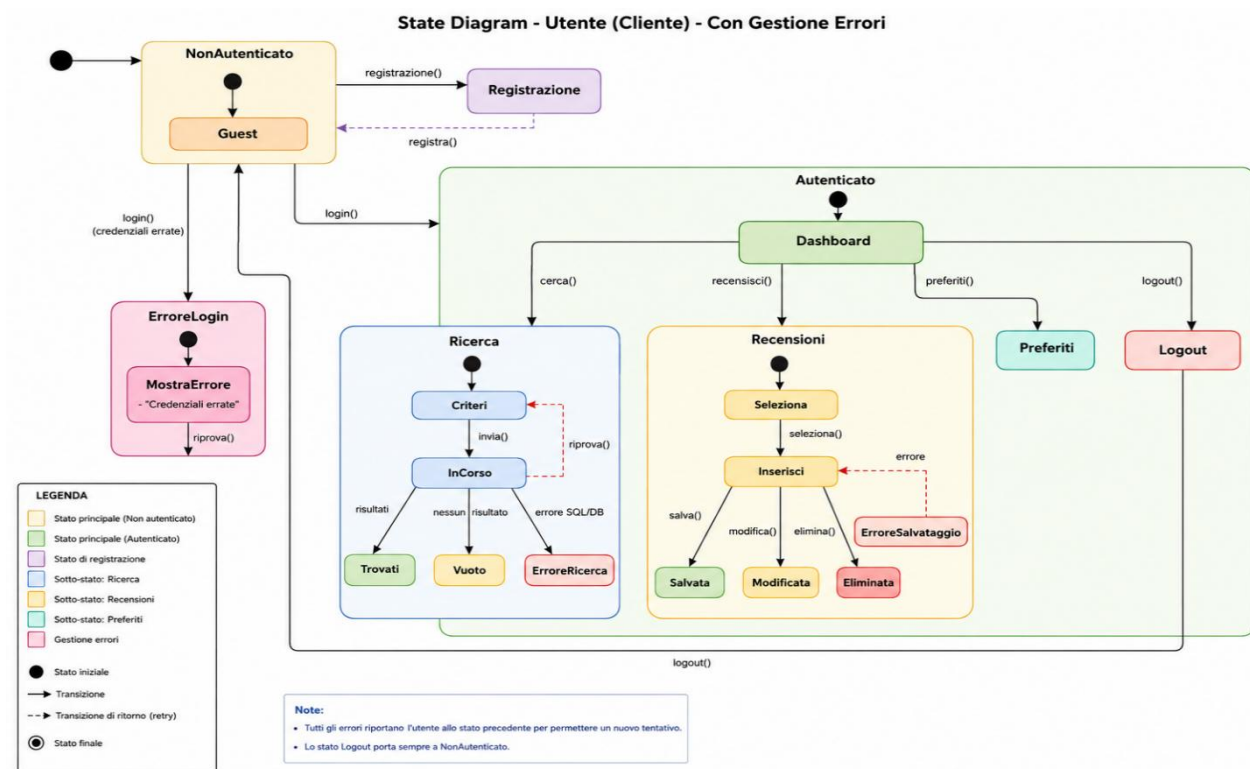
4.3.4 Sequence - Ricerca Vicini:

Descrizione:

Il Sequence Diagram rappresenta il flusso della ricerca dei ristoranti vicini. Dopo che l'utente inserisce il luogo e il raggio di ricerca, il client invia la richiesta allo SlaveThread, che recupera le coordinate della città e ricerca i ristoranti presenti nel database. Per ciascun ristorante viene calcolata la distanza tramite il componente GeoUtils; vengono quindi selezionati solo quelli compresi nel raggio indicato, ordinati per distanza e restituiti al client. Se il luogo non viene trovato, il sistema restituisce un messaggio di errore all'utente.



4.4.1 State Diagram:



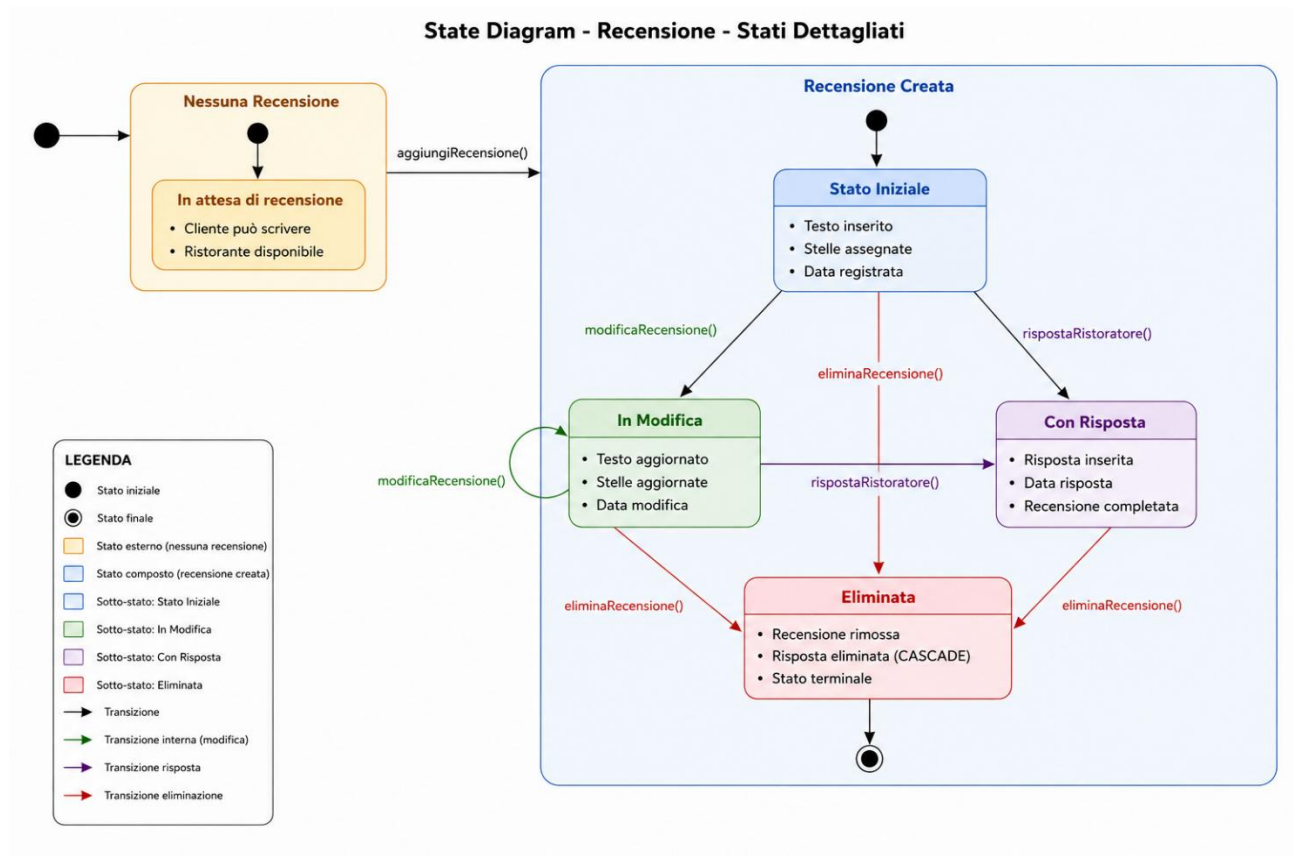
Il diagramma degli stati descrive le transizioni del client tra le diverse schermate (login, pannelli specifici del ruolo, ricerca avanzata, dettaglio ristorante, operazioni autenticate).

4.4.2 State Diagram della recensione:

Descrizione:

Lo State Diagram descrive il ciclo di vita di una recensione all'interno del sistema TheKnife.

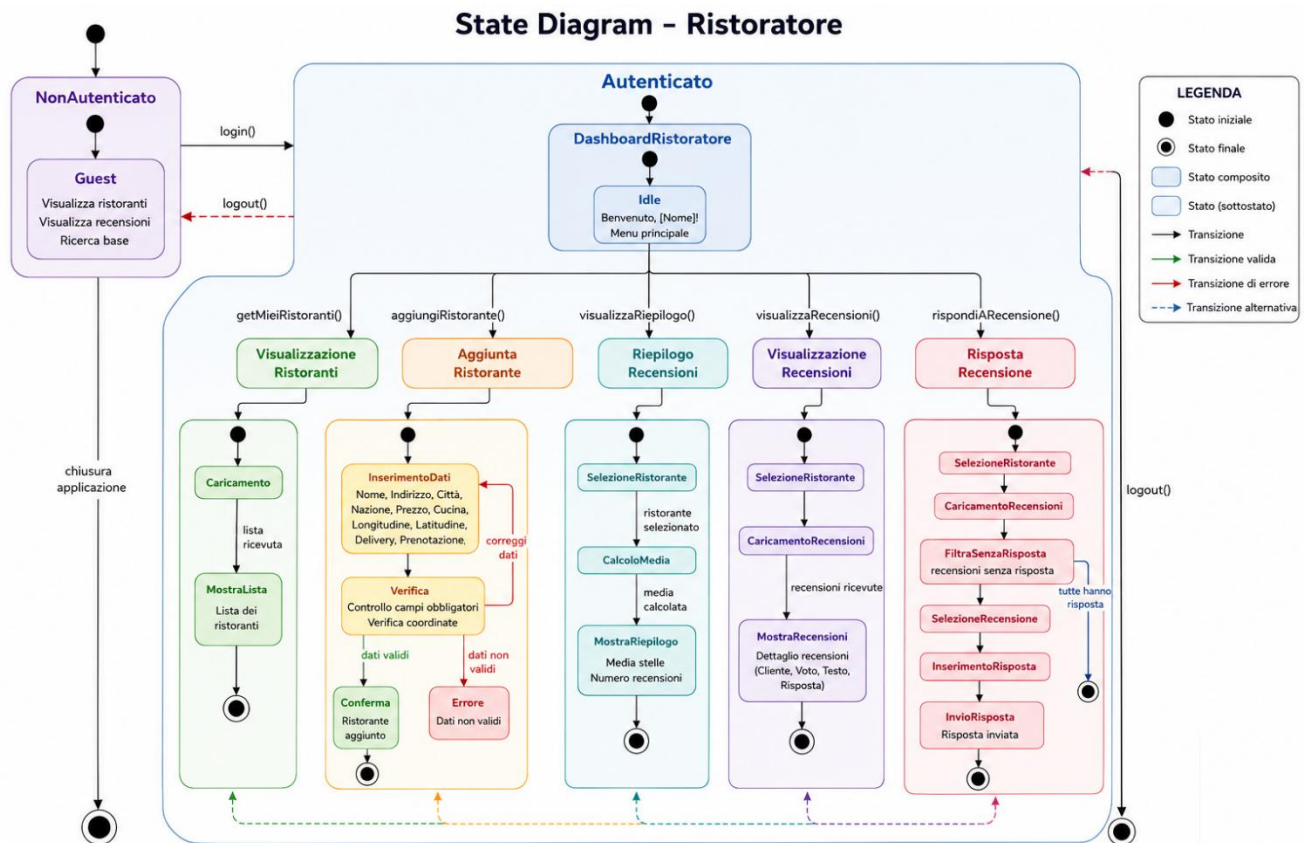
Una recensione nasce quando il cliente la inserisce, può essere modificata, ricevere una risposta dal ristoratore oppure essere eliminata. Ogni transizione rappresenta un'operazione disponibile nel sistema, mentre lo stato finale indica la rimozione definitiva della recensione e dell'eventuale risposta associata tramite eliminazione a cascata (CASCADE).



4.4.3 State Diagram-Ristoratore:

Descrizione:

Lo State Diagram descrive il comportamento del ristoratore all'interno del sistema TheKnife. Dopo l'autenticazione, il ristoratore accede alla dashboard dalla quale può visualizzare i propri ristoranti, aggiungerne di nuovi, consultare il riepilogo delle recensioni, visualizzare i commenti ricevuti e rispondere alle recensioni ancora prive di risposta. Ogni funzionalità è rappresentata come uno stato con le relative transizioni, fino al logout o alla chiusura dell'applicazione.

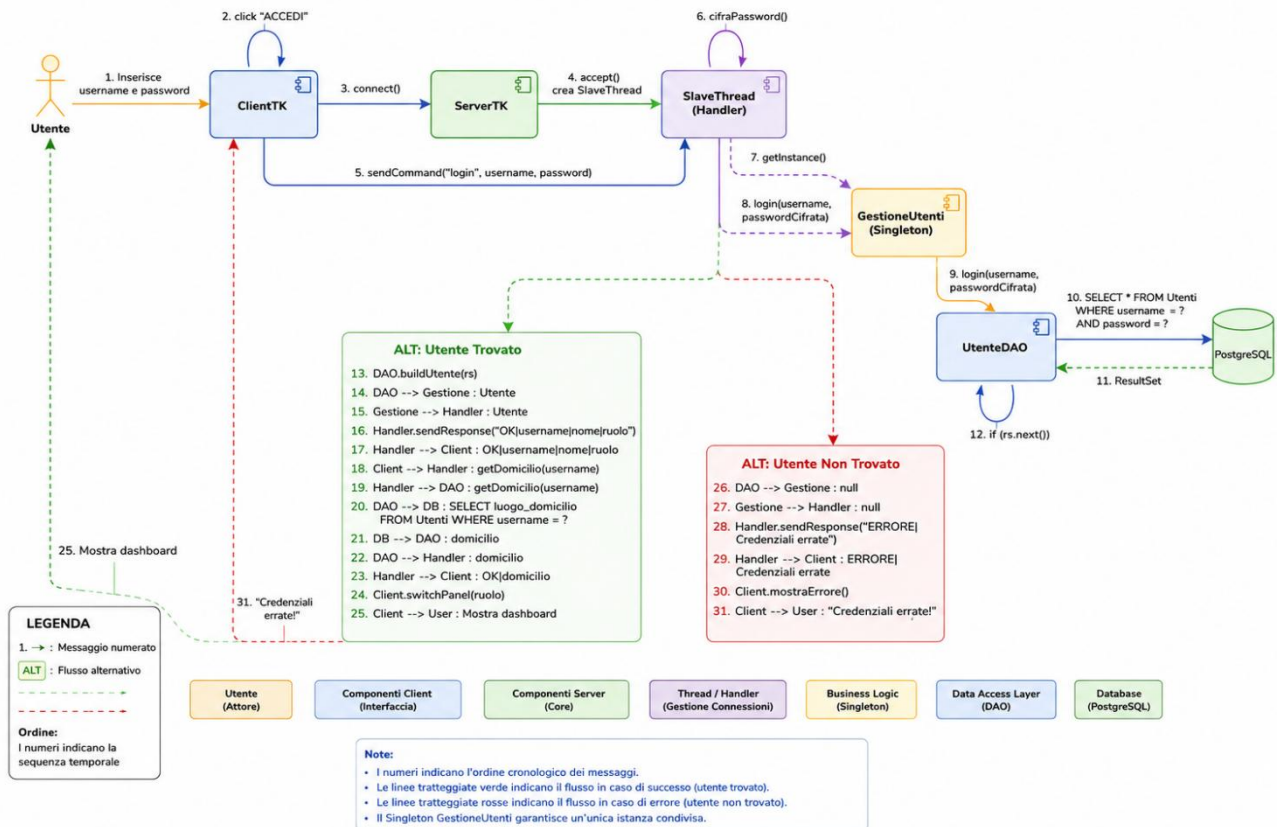


4.5 Communication Diagram:

Il Communication Diagram rappresenta lo scambio di messaggi tra i componenti coinvolti nel processo di autenticazione dell'utente. Il client invia la richiesta di login al server, che la gestisce tramite uno SlaveThread e il componente GestioneUtenti. Successivamente viene interrogato il UtenteDAO, che verifica le credenziali nel database PostgreSQL. In base all'esito della verifica, il sistema restituisce al client le informazioni dell'utente autenticato oppure un messaggio di errore, mostrando così i due possibili flussi di esecuzione del login.

- Nota: Le istruzioni per l'uso dell'applicazione, complete di screenshot e guide passo-passo, sono disponibili nel Manuale Utente(manuale_utente.pdf), documento separato che accompagna la presente documentazione tecnica.

Communication Diagram - Login - Con Dettaglio Messaggi



5. Pattern Utilizzati:

Durante lo sviluppo sono stati applicati diversi design pattern per migliorare la manutenibilità, la chiarezza del codice e la corretta separazione delle responsabilità. Di seguito vengono descritti i pattern principali con esempi di codice tratti dal progetto.

5.1 Singleton (GestioneUtenti):

Il pattern Singleton è applicato alla classe GestioneUtenti per garantire che esista una sola istanza condivisa tra tutti i thread del server. Il metodo getInstance() è sincronizzato per garantire la thread-safety.

```
public class GestioneUtenti {
    private static GestioneUtenti instance;

    private GestioneUtenti() { /* costruttore privato */ }

    public static synchronized GestioneUtenti getInstance() {
        if (instance == null) {
            instance = new GestioneUtenti();
        }
        return instance;
    }
}
```

Vantaggi:

- Evita la creazione di istanze multiple e garantisce coerenza nello stato.
- Accesso controllato a risorse condivise.
- Thread-safe grazie alla sincronizzazione del metodo factory.

5.2 Factory Method (UtenteDAO.buildUtente):

Il pattern Factory Method viene utilizzato in UtenteDAO.buildUtente() per creare l'oggetto Utente appropriato (Cliente o Ristoratore) in base al valore del campo ruolo restituito dal database. In questo modo le classi chiamanti non devono conoscere la gerarchia delle sottoclassi.

```
private static Utente buildUtente(ResultSet rs) throws SQLException {
    String ruolo = rs.getString("ruolo");
    if ("cliente".equalsIgnoreCase(ruolo)) {
        return new Cliente(rs.getString("nome"), rs.getString("cognome"),
                           rs.getString("username"), rs.getString("password"),
                           rs.getString("data_nascita"),
                           rs.getString("luogo_domicilio"));
    } else {
        return new Ristoratore(rs.getString("nome"), rs.getString("cognome"),
                               rs.getString("username"), rs.getString("password"),
                               rs.getString("data_nascita"),
                               rs.getString("luogo_domicilio"));
    }
}
```

5.3 DAO (Data Access Object):

Il pattern DAO disaccoppia il codice di accesso al database dalla logica applicativa. Ogni classe DAO (UtenteDAO, RistoranteDAO, RecensioneDAO) espone metodi ad alto livello che incapsulano le query JDBC, permettendo di cambiare il backend di persistenza senza modificare il resto dell'applicazione.

```
public class RistoranteDAO {
    public static ArrayList<RistoranteDTO> cercaTuttiCriteri(
        String luogo, String cucina,
        Double prezzoMin, Double prezzoMax,
        Boolean delivery, Boolean prenotazione,
        Double mediaMin) throws SQLException {
        // Costruisce dinamicamente la query e ritorna i DTO
    }
}
```

5.4 DTO (Data Transfer Object):

La classe RistoranteDTO è un Data Transfer Object: una struttura serializzabile, priva di logica di business, utilizzata per trasferire dati tra server e client attraverso ObjectOutputStream / ObjectInputStream

```
public class RistoranteDTO implements Serializable {
    private static final long serialVersionUID = 1L;
    private String nome, indirizzo, citta, nazione, prezzo, tipoCucina;
    private double latitudine, longitudine;
    private boolean delivery, prenotazione;
    private double mediaStelle;
    private int numRecensioni;
    // getter/setter ...
}
```


5.5 Observer (ActionListener in Swing):

Tutta l'interfaccia Swing è basata sul pattern Observer: la classe ClientTK implementa ActionListener e si registra sui bottoni (subject). Quando l'utente preme un pulsante, viene notificata e il metodo actionPerformed() viene invocato.

```
public class ClientTK extends JFrame implements ActionListener {
    private JButton btnLogin = new JButton("Login");

    public ClientTK() {
        btnLogin.addActionListener(this);    // registrazione observer
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == btnLogin) {
            eseguiLogin();
        }
    }
}
```

5.6 SwingWorker:

Per non bloccare l'Event Dispatch Thread (EDT) durante le chiamate di rete al server, le richieste vengono incapsulate in SwingWorker. Il metodo doInBackground() viene eseguito in un thread separato, mentre done() aggiorna la GUI sull'EDT.

```
new SwingWorker<Object, Void>() {
    @Override
    protected Object doInBackground() throws Exception {
        return sendCommand("cercaMultiCriterio", luogo, cucina, /* ... */);
    }
    @Override
    protected void done() {
        try {
            mostraRisultati((ArrayList<RistoranteDTO>) get());
        } catch (Exception ex) { mostraErrore(ex); }
    }
}.execute();
```

5.7 Thread Pool (ExecutorService):

Per gestire le connessioni dei client in modo efficiente, il server utilizza un **Thread Pool** implementato tramite `ExecutorService.newCachedThreadPool()`.

Questa soluzione evita la creazione di un nuovo thread per ogni connessione in ingresso, riducendo il costo di creazione e distruzione dei thread e migliorando le prestazioni complessive del sistema. Ogni nuova richiesta viene infatti assegnata a un thread disponibile del pool oppure, se necessario, a un nuovo thread creato automaticamente.

```
ExecutorService pool = Executors.newCachedThreadPool();

while (running) {
```

```

        Socket sock = serverSocket.accept();
        System.out.println("Connessione accettata da " + sock.getInetAddress());
        pool.execute(new SlaveThread(sock));
    }

```

Vantaggi dell'approccio

Vantaggio	Descrizione
Riutilizzo dei thread	I thread già creati vengono riutilizzati, riducendo l'overhead dovuto alla loro continua creazione e distruzione.
Gestione automatica	Il pool crea nuovi thread solo quando necessario e riutilizza quelli già disponibili.
Scalabilità	Consente di gestire un elevato numero di client concorrenti mantenendo buone prestazioni.
Ottimizzazione delle risorse	I thread inutilizzati vengono automaticamente eliminati dopo un periodo di inattività, limitando il consumo di memoria.

Arresto controllato del Thread Pool

Durante lo spegnimento del server viene eseguita una procedura di chiusura controllata tramite uno **Shutdown Hook**, che consente di terminare correttamente tutti i thread ancora attivi e di rilasciare le risorse utilizzate.

```

Runtime.getRuntime().addShutdownHook(new Thread(() -> {
    running = false;
    pool.shutdown();
    try {
        if (!pool.awaitTermination(10, TimeUnit.SECONDS)) {
            pool.shutdownNow();
        }
    } catch (InterruptedException e) {
        pool.shutdownNow();
    }
    DataBaseManager.closePool();
}));

```

Questa procedura garantisce che:

- i thread ancora in esecuzione possano completare le operazioni già avviate;
- il Thread Pool venga chiuso entro un tempo massimo di **10 secondi**;
- eventuali thread rimasti attivi vengano terminati in modo forzato;
- tutte le connessioni al database vengano chiuse correttamente, evitando perdite di risorse.

6. Strutture Dati:

Le strutture dati utilizzate nell'applicazione sono state scelte in funzione delle specifiche operazioni richieste:

Struttura	Impiego	Motivazione
ArrayList<RistoranteDTO>	Risultati di ricerca, lista preferiti, lista ristoranti del ristorante.	Accesso O(1) per indice e iterazione efficiente per la JList Swing.
ArrayList<Recensione>	Recensioni di un ristorante e del cliente loggato.	Ordine di inserimento preservato, costo costante in append.
HashMap<String, Object>	Cache temporanea della sessione client (utente loggato, domicilio).	Lookup O(1) per chiave.
DefaultListModel<String>	Modello dati delle JList della GUI.	Notifica automatica della view quando i dati cambiano (Observer).
ResultSet (JDBC)	Lettura dei dati dal database.	Streaming row-by-row, evita di caricare tutto in memoria.
String dinamico (StringBuilder)	Costruzione query SQL multi-criterio.	Concatenazione efficiente delle clausole WHERE.
ExecutorService (Thread Pool)	Server-side: gestione concorrente dei client.	Riuso dei thread, limiti di concorrenza configurabili.

7. Scelte Algoritmiche:

7.1 Cifratura Password (SHA-256):

Le password non sono mai salvate in chiaro: vengono cifrate lato server con l'algoritmo SHA-256 prima dell'inserimento in tabella. Lo stesso hash viene confrontato durante il login. SHA-256 produce un digest di 256 bit (64 caratteri esadecimali) ed è praticamente impossibile da invertire.

```
public static String cifraPassword(String password) {
    try {
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        byte[] hash = md.digest(password.getBytes(StandardCharsets.UTF_8));
        StringBuilder sb = new StringBuilder();
        for (byte b : hash) sb.append(String.format("%02x", b));
        return sb.toString();
    } catch (NoSuchAlgorithmException e) {
        throw new RuntimeException(e);
    }
}
```

7.2 Calcolo Distanza (Haversine):

Per la ricerca dei ristoranti "vicini" ad una posizione, viene impiegata la formula di Haversine, che calcola la distanza in chilometri tra due punti dati in latitudine e longitudine, considerando la Terra come una sfera di raggio $R = 6371$ km.

```

public static double distanzaKm(double lat1, double lon1,
                                double lat2, double lon2) {
    final double R = 6371.0; // raggio terrestre [km]
    double dLat = Math.toRadians(lat2 - lat1);
    double dLon = Math.toRadians(lon2 - lon1);
    double a = Math.sin(dLat/2) * Math.sin(dLat/2) +
                Math.cos(Math.toRadians(lat1)) *
                Math.cos(Math.toRadians(lat2)) *
                Math.sin(dLon/2) * Math.sin(dLon/2);
    double c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
    return R * c;
}

```

Complessità: $O(1)$ per coppia di punti. L'ordinamento dei risultati per distanza è $O(n \log n)$.

7.3 Ricerca Multi-Criterio:

L'algoritmo di ricerca multi-criterio costruisce dinamicamente la query SQL aggiungendo una clausola WHERE per ogni filtro non nullo. Il filtro sulla media stelle è applicato tramite HAVING su una GROUP BY r.id. La conversione del prezzo simbolico in numerico è incapsulata in un'espressione CASE per supportare l'intervallo numerico.

```

StringBuilder q = new StringBuilder(
    "SELECT r.*, " +
    "  COALESCE(AVG(rec.stelle), 0) AS media, " +
    "  COUNT(rec.id) AS num_rec " +
    "FROM RistorantiTheKnife r " +
    "LEFT JOIN Recensioni rec ON rec.ristorante_id = r.id " +
    "WHERE r.citta ILIKE ? ");
if (cucina != null) q.append("AND r.tipo_cucina ILIKE ? ");
if (delivery != null) q.append("AND r.delivery = ? ");
if (prenotazione != null) q.append("AND r.prenotazione = ? ");
if (prezzoMin != null || prezzoMax != null) {
    q.append("AND (CASE WHEN r.prezzo ~ '^([0-9]+(\\.[0-9]+)?$' " +
        "      THEN r.prezzo::DOUBLE PRECISION " +
        "      ELSE LENGTH(TRIM(r.prezzo)) * 37.5 END) " +
        "BETWEEN ? AND ? ");
}
q.append("GROUP BY r.id ");
if (mediaMin != null) q.append("HAVING AVG(rec.stelle) >= ? ");

```

8. Gestione della Concorrenza:

L'applicazione è progettata per supportare l'interazione parallela con più utenti connessi da postazioni differenti. La gestione della concorrenza è realizzata su più livelli:

Livello server (thread-per-client)

- Il server crea un nuovo SlaveThread per ogni client che si connette.
- Viene utilizzato un ExecutorService (thread pool) per limitare il numero massimo di thread attivi.

- Ogni `SlaveThread` mantiene il proprio socket, gli stream e lo stato della sessione

```
ExecutorService pool = Executors.newCachedThreadPool();
while (true) {
    Socket sock = serverSocket.accept();
    pool.execute(new SlaveThread(sock));
}
```

Livello applicativo:

- La classe Singleton `GestioneUtenti` è acceduta tramite metodo `getInstance()` sincronizzato.
- Le risorse condivise sono accedute con blocchi `synchronized` dove necessario.

Livello database:

- PostgreSQL gestisce nativamente la concorrenza con il modello MVCC (Multi-Version Concurrency Control).
- I vincoli `UNIQUE` garantiscono l'integrità anche in presenza di `INSERT` concorrenti, supportati dalla clausola `ON CONFLICT DO NOTHING`.
- Le connessioni sono ottenute on-demand da `DataBaseManager` e rilasciate al termine di ogni operazione (`try-with-resources`).

Livello client (EDT vs background):

- L'Event Dispatch Thread (EDT) Swing non viene mai bloccato.
- Tutte le chiamate di rete sono incapsulate in `SwingWorker`; gli aggiornamenti GUI avvengono in `done()` o `publish()`.

9. Interfaccia Grafica:

L'interfaccia grafica di **TheKnife** è stata sviluppata utilizzando la libreria **Java Swing** e adotta un'architettura basata su **CardLayout**, che consente di gestire la navigazione tra le diverse schermate dell'applicazione senza aprire nuove finestre. Questa scelta garantisce una navigazione fluida, una migliore organizzazione dell'interfaccia e un'esperienza d'uso più intuitiva.

Palette dei colori

Per garantire uniformità grafica e una migliore esperienza utente è stata definita una palette di colori coerente, utilizzata in tutta l'applicazione.

Colore	Codice	Utilizzo
--------	--------	----------

Rosa Primario	#C8506E	Pulsanti principali, titoli ed elementi di rilievo
---------------	---------	--

Rosa Secondario	#DC96AF	Pulsanti secondari e componenti decorativi
-----------------	---------	--

Colore	Codice	Utilizzo
Rosa Chiaro	#FCEBF0	Sfondo principale dell'applicazione
Rosa Gradiente	#FFDCE6	Gradiente utilizzato nelle schermate principali
Grigio Scuro	#3C3741	Testi principali
Grigio Medio	#8C8791	Testi secondari
Bianco	#FFFFFF	Card, pannelli e campi di input

Schermate principali:

L'applicazione è composta dalle seguenti schermate principali:

- **Login:** consente all'utente di autenticarsi tramite username e password oppure di accedere come ospite o registrarsi.
- **Registrazione:** permette la creazione di un nuovo account inserendo i dati personali, il ruolo (Cliente o Ristoratore) e le credenziali di accesso.
- **Guest:** schermata dedicata agli utenti non autenticati, dalla quale è possibile consultare i ristoranti ed effettuare una ricerca di base.
- **Ricerca Avanzata:** consente di filtrare i ristoranti utilizzando diversi criteri, tra cui città, tipo di cucina, fascia di prezzo, servizio di delivery, prenotazione online, valutazione minima e ricerca per vicinanza.
- **Dashboard Cliente:** mette a disposizione tutte le funzionalità dedicate ai clienti, come la ricerca dei ristoranti, la gestione dei preferiti, la gestione delle recensioni e la ricerca dei ristoranti vicini al proprio domicilio.
- **Dashboard Ristoratore:** permette al ristoratore di gestire i propri ristoranti, aggiungerne di nuovi, consultare il riepilogo delle recensioni, visualizzare i commenti ricevuti e rispondere alle recensioni dei clienti.

Caratteristiche dell'interfaccia

L'interfaccia è stata progettata seguendo uno stile moderno e uniforme, adottando i seguenti elementi grafici:

- pulsanti con angoli arrotondati, ombreggiatura ed effetto **hover**;
- campi di testo con bordi arrotondati per migliorare la leggibilità;
- card centrali con effetto ombra per evidenziare le informazioni principali;
- **Look and Feel FlatLaf**, scelto per ottenere un aspetto moderno, pulito e professionale.

Gestione della concorrenza nella GUI:

Per mantenere l'interfaccia sempre reattiva, tutte le operazioni che richiedono comunicazioni con il server vengono eseguite in background mediante **SwingWorker**. In questo modo l'**Event Dispatch Thread (EDT)** non viene bloccato durante operazioni potenzialmente lunghe, come le richieste di rete o l'accesso al database, consentendo all'utente di continuare a interagire con l'applicazione senza rallentamenti o blocchi dell'interfaccia.

10. JavaDoc:

L'intero codice sorgente è commentato in formato JavaDoc, parte integrante del Manuale Tecnico secondo le linee guida del corso. Ogni package, classe, interfaccia, attributo e metodo è documentato con un blocco `/** ... */` che descrive scopo, parametri, valore di ritorno ed eccezioni.

Tag utilizzati

Tag	Descrizione
@author	Indica l'autore (uno per riga se più autori).
@param	Descrizione di un parametro (in ordine di dichiarazione).
@return	Valore restituito dal metodo.
@throws	Eccezione che il metodo può sollevare e relative cause.
@see	Riferimento incrociato ad altre classi/metodi.
@version	Versione del componente.

Esempio di documentazione di classe e metodo:

```
/**
 * Singleton che gestisce la registrazione, il login e la cifratura
 * delle password degli utenti di TheKnife.
 *
 * @author Omema Gharsellaoui
 * @author Razak Kufura
 * @version 1.0
 */
public class GestioneUtenti {

    /**
     * Restituisce l'unica istanza condivisa della classe.
     *
     * @return l'istanza singleton di {@code GestioneUtenti}.
     */
    public static synchronized GestioneUtenti getInstance() { /* ... */ }

    /**
     * Cifra la password con l'algoritmo SHA-256.
     *
     * @param password password in chiaro
     * @return digest esadecimale di 64 caratteri
     * @throws RuntimeException se l'algoritmo SHA-256 non è disponibile
     */
    public static String cifraPassword(String password) { /* ... */ }
}
```

Generazione della documentazione:

Per generare l'intera documentazione HTML è sufficiente lanciare:

```
mvn javadoc:javadoc
```

La documentazione viene prodotta nella cartella `target/site/apidocs/` e può essere navigata aprendo il file `index.html`.

11. Bibliografia e Sitografia

Documenti del corso:

- G. Meroni, "Laboratorio Interdisciplinare B — Specifiche del Progetto TheKnife", Università degli Studi dell'Insubria, A.A. 2024/2025.
- G. Meroni, "Laboratorio Interdisciplinare B — Documentazione di Progetto", Università degli Studi dell'Insubria, A.A. 2024/2025.

Testi e risorse tecniche:

- E. Gamma, R. Helm, R. Johnson, J. Vlissides, "Design Patterns: Elements of Reusable Object-Oriented Software", Addison-Wesley, 1994.
- R. Elmasri, S. B. Navathe, "Sistemi di basi di dati. Fondamenti", Pearson, ediz. corrente.
- J. Bloch, "Effective Java", 3rd Edition, Addison-Wesley.
- "The Java™ Tutorials — Creating a GUI With Swing", Oracle. URL: <https://docs.oracle.com/javase/tutorial/uiswing/>

Strumenti e tecnologie:

- PostgreSQL — <https://www.postgresql.org/>
- JDBC API Reference — <https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/>
- Apache Maven — <https://maven.apache.org/>

Librerie e framework utilizzati:

- HikariCP — Pool di connessioni JDBC ad alte prestazioni.

URL: <https://github.com/brettwooldridge/HikariCP>

- FlatLaf — Look and Feel moderno per interfacce Swing.

URL: <https://github.com/JFormDesigner/FlatLaf>

- SLF4J — Simple Logging Facade for Java.

URL: <https://www.slf4j.org/>

Dataset :

- "Michelin Guide Restaurants 2021", Kaggle dataset (ngshiheng). URL: <https://www.kaggle.com/datasets/ngshiheng/michelin-guide-restaurants-2021>